

# claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_49c1018a-e67\agent-a3fd2ee1f8221dc96.jsonl`  
- SHA-256: `ece0c41c6ef6fd625a15b415279bc181d1d2f439549293b1cd76c60b0997ee8f`  
- Source modified: `2026-06-18T01:41:37+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_49c1018a-e67`  
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

## Transcript

### 1. user (2026-06-18T01:40:32.121Z)

Adversarial bug-hunter, mature Python repo (badminton-highlight-indexer). Read READ-ONLY in this worktree (origin/master), DO NOT modify:

```
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base  
A prior precision pass found ZERO bugs; you cross-cut by BUG CLASS across ALL files to catch what an area-by-area pass missed. Report a finding ONLY if you READ the code and a CONCRETE trigger reaches it and a maintainer would call it a real defect. Include exact file+line, the trigger, why it's wrong, severity, minimal fix, confidence. PRECISION over recall ? empty list is a valid, respected answer; do NOT manufacture findings. EXCLUDE backend/eval/ablation.py, backend/eval/fusion_compare.py, backend/eval/fusion_audio.py, scripts/nightly_regression.py.
```

CANDIDATE:

```
{  
  "title": "Unguarded b[\"precision\"]/b[\"recall\"] on a persisted (committed, hand-editable) regression-baseline record",  
  "file": "backend/eval/regression.py",  
  "line": "129",  
  "trigger": "regress() loads the tracked eval_baselines/regression_baseline.json via load_baselines() (plain json.load, no validation) and looks up the record b = baselines.get(\"<video_id>::<stage>\"). If that file contains a baseline entry missing the \"precision\" or \"recall\" key ? e.g. a hand-edited entry, a partial/older-schema record, or one produced/merged outside save_baseline ? line 129 `base_p, base_r = b[\"precision\"], b[\"recall\"]` raises KeyError and aborts the regression run.",  
  "why_wrong": "b is a record deserialized from a persisted, explicitly TRACKED-and-committable JSON artifact (the module docstring/constant call it out as a checked-in file CI loads). Every OTHER field on the very same record is read defensively on the adjacent lines (line 132: b.get(\"iou_threshold\"), b.get(\"detector\"), b.get(\"gt_hash\")), and the function is clearly designed to degrade gracefully around baselines (has_baseline flag, the 'no baseline recorded' branch, the incommensurable-comparison reasons). Reading the two metric fields with bare [...] indexing is an inconsistency that turns a malformed/partial baseline entry into an uncaught KeyError crash instead of a handled condition.",  
  "severity": "low",  
  "proposed_fix": "Read the metric fields defensively like the sibling fields and treat a record lacking them as having no usable baseline. E.g.: `base_p, base_r = b.get(\"precision\"), b.get(\"recall\")` and guard the two regression comparisons with `if base_p is not None and s.precision < base_p - tolerance:` / `if base_r is not None and ...`; when either is missing, append a reason like 'baseline record missing precision/recall (malformed)' and skip the metric comparison (optionally set has_baseline=False) rather than KeyError.",  
  "confidence": 0.5,  
  "klass": "unguarded access on EXTERNAL or PERSISTED data"  
}
```

Read C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base/backend/eval/regression.py and related code/tests READ-ONLY before judging.

LENS ? REACHABILITY: prove a real path reaches it, or set real=false.

## 2. assistant (2026-06-18T01:40:37.111Z)

I'll analyze this candidate bug by reading the target file and related code.

## 3. user (2026-06-18T01:40:37.678Z)

```
1      """Regression runner ? verify the production tool against golden-set ground truth.
2
3      Implements the explicit ask: *"given a video, get a golden set created and verify
4      its precision and recall against the production tool."* This is deliberately a thin
5      wrapper over machinery that already exists:
6
7      ground truth  ? AnnotationProvider (file / vendor / oracle) [backend.annotations]
8      predictions   ? the production pipeline's stored DB rows    [backend.eval.harness]
9      comparison    ? interval matching + P/R/F1                  [backend.eval.metrics]
10     baseline      ? a per-video snapshot of "known good" numbers [this module]
11
12     A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13     baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14     later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15     independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16     """
17     import os
18     import json
19     import hashlib
20     import logging
21     from dataclasses import dataclass, asdict
22     from typing import Dict, List, Optional
23
24     from backend.infrastructure.database import Database
25     from backend.eval import metrics
26     from backend.eval.harness import get_predictions
27     from backend.annotations import annotation_registry
28
29     logger = logging.getLogger(__name__)
30
31     # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32     # actually load a committed baseline to compare against.
33     DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
34     DEFAULT_TOLERANCE = 0.05
35
36
37     def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38         """Stable short hash of the GT intervals ? detects a baseline taken against
39         different labels."""
40         norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
41         return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]
42
43     @dataclass
44     class RegressionResult:
45         video_id: str
46         stage: str
47         iou_threshold: float
48         precision: float
49         recall: float
50         f1: float
51         # Baseline values compared against (None when no baseline existed yet).
52         baseline_precision: Optional[float]
53         baseline_recall: Optional[float]
54         tolerance: float
55         regressed: bool
56         reasons: List[str]
```

```

57         # True only when a baseline existed to compare against. Distinguishes a genuine PASS
58         # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
59         has_baseline: bool = False
60         gt_hash: Optional[str] = None
61
62         def as_dict(self) -> dict:
63             return asdict(self)
64
65
66         # ----- baseline store
67
68         def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
69             """Load the per-video baseline snapshot file (returns {} if absent)."""
70             if not os.path.isfile(path):
71                 return {}
72             with open(path) as f:
73                 return json.load(f)
74
75
76         def _baseline_key(video_id: str, stage: str) -> str:
77             return f"{video_id}::{stage}"
78
79
80         def save_baseline(video_id: str, stage: str, precision: float, recall: float,
81                           f1: float, path: str = DEFAULT_BASELINE_PATH,
82                           iou_threshold: Optional[float] = None, detector: Optional[str] = None,
83                           gt_fingerprint: Optional[str] = None) -> None:
84             """Snapshot a video/stage's current numbers as the new 'known good' baseline.
85
86             Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
87             later run computed under different settings (or against different labels) is
detected
88             as incommensurable rather than silently compared.
89             """
90             baselines = load_baselines(path)
91             baselines[_baseline_key(video_id, stage)] = {
92                 "video_id": video_id, "stage": stage,
93                 "precision": precision, "recall": recall, "f1": f1,
94                 "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
95             }
96             os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
97             with open(path, "w") as f:
98                 json.dump(baselines, f, indent=2, sort_keys=True)
99             logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
100 _baseline_key(video_id, stage), precision, recall)
101
102         # ----- the runner
103
104         def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
105                     stage: str = "final", iou_threshold: float = 0.5,
106                     detector: Optional[str] = None,
107                     tolerance: float = DEFAULT_TOLERANCE,
108                     baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
109             """Score stored predictions for one video against ground truth and compare to
baseline.
110
111             `ground_truth` is a list of (start, end) intervals ? typically obtained from an
112             AnnotationProvider (see `regress_with_provider`). A regression is flagged when
113             precision OR recall falls more than `tolerance` below the snapshotted baseline.
114             If no baseline exists yet, `regressed` is False (nothing to compare to) and the
115             caller can choose to snapshot the current numbers via `save_baseline`.
116             """
117             preds = get_predictions(db, video_id, stage, detector=detector)

```

```

118     s = metrics.score(preds, ground_truth, iou_threshold)
119     fp = gt_hash(ground_truth)
120
121     baselines = load_baselines(baseline_path)
122     b = baselines.get(_baseline_key(video_id, stage))
123
124     reasons: List[str] = []
125     regressed = False
126     has_baseline = b is not None
127     base_p = base_r = None
128     if b is not None:
129         base_p, base_r = b["precision"], b["recall"]
130         # Incommensurable comparison guard: a baseline taken at a different IoU/detector
or
131         # against different labels is not a valid reference ? flag it instead of
trusting it.
132         b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
b.get("gt_hash")
133         if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
134             regressed = True
135             reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
(incommensurable)")
136         if b_det is not None and b_det != detector:
137             regressed = True
138             reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
139         if b_hash is not None and b_hash != fp:
140             regressed = True
141             reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
142         if s.precision < base_p - tolerance:
143             regressed = True
144             reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
145         if s.recall < base_r - tolerance:
146             regressed = True
147             reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
148     else:
149         reasons.append("no baseline recorded for this video/stage ? nothing to compare")
150
151     return RegressionResult(
152         video_id=video_id, stage=stage, iou_threshold=iou_threshold,
153         precision=s.precision, recall=s.recall, f1=s.f1,
154         baseline_precision=base_p, baseline_recall=base_r,
155         tolerance=tolerance, regressed=regressed, reasons=reasons,
156         has_baseline=has_baseline, gt_hash=fp,
157     )
158
159
160 def regress_with_provider(db: Database, video_id: str, video_ref: str,
161                           tier: str = "file", stage: str = "final",
162                           iou_threshold: float = 0.5,
163                           detector: Optional[str] = None,
164                           tolerance: float = DEFAULT_TOLERANCE,
165                           baseline_path: str = DEFAULT_BASELINE_PATH,
166                           guideline: Optional[str] = None,
167                           sport: str = "badminton",
168                           provider_kwargs: Optional[dict] = None) -> RegressionResult:
169     """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
170
171     `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
once those providers land). `video_ref` is whatever that provider expects (a CSV
path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored

```

```

174     predictions in the DB (the file MD5).
175     """
176     provider = annotation_registry.get(tier, **(provider_kwargs or {}))
177     gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
178     return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
179                   detector=detector, tolerance=tolerance, baseline_path=baseline_path)
180
181
182     def format_result(r: RegressionResult) -> str:
183         verdict = "REGRESSED" if r.regressed else "ok"
184         head = (f"[{verdict}] {r.video_id} ({r.stage}) "
185               f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
186         if r.baseline_precision is not None:
187             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
188                   tol={r.tolerance}"
189         return head + " ".join(f"\n    - {why}" for why in r.reasons)

```

#### 4. assistant (2026-06-18T01:40:40.196Z)

Let me examine the callers of `regress()` and the baseline file to understand reachability, plus the tests.

#### 5. user (2026-06-18T01:40:42.244Z)

```

.claude\worktrees\hunt-base\run_regression.py:62:     result = regression.regress_with_provider(
.claude\worktrees\hunt-base\run_regression.py:71:
regression.save_baseline(result.video_id, result.stage, result.precision,
.claude\worktrees\hunt-base\docs\CODE_MAP.md:98:     -> regression.regress(_with_provider) vs
baseline @backend/eval/regression.py [CI gate]
.claude\worktrees\hunt-base\docs\CODE_MAP.md:122:- `@run_regression.py::main` ?
`regress_with_provider` vs baseline. Exit codes semantic: **0=ok, 1=REGRESSION (CI gate),
2=missing DB**. ? `--update-baseline` runs AFTER compare (snapshots even a regressing run);
`::default_db_path` resolves via `backend.config.load_config`.
.claude\worktrees\hunt-base\docs\CODE_MAP.md:180:[Omitted long matching line]
.claude\worktrees\hunt-base\docs\BACKLOG.md:46:- ? **GS-5 ? regression triggers** ? wire
`regress()` into CI (pre-merge gate), on-demand CLI (the "surprise" case), and a scheduled run;
include a deliberate re-baseline path. **Revisit:** after GS-4.
.claude\worktrees\hunt-base\scripts\nightly_regression.py:509:def save_baseline(path: str,
snapshot: Dict[str, Any], tols: Tolerances) -> None:
.claude\worktrees\hunt-base\scripts\nightly_regression.py:567:
save_baseline(args.baseline, snapshot, tols)
.claude\worktrees\hunt-base\docs\archives\decisions\DECISIONS.md:274:4. **Regression = existing
harness + one new component.** `regress(video)` calls a provider for ground truth, runs the
production pipeline, and scores with the unchanged `metrics.py` against a per-video
`baseline.json`.
.claude\worktrees\hunt-base\backend\eval\regression.py:68:def load_baselines(path: str =
DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
.claude\worktrees\hunt-base\backend\eval\regression.py:80:def save_baseline(video_id: str,
stage: str, precision: float, recall: float,
.claude\worktrees\hunt-base\backend\eval\regression.py:90:     baselines = load_baselines(path)
.claude\worktrees\hunt-base\backend\eval\regression.py:104:def regress(db: Database, video_id:
str, ground_truth: List[metrics.Interval],
.claude\worktrees\hunt-base\backend\eval\regression.py:112:     AnnotationProvider (see
`regress_with_provider`). A regression is flagged when
.claude\worktrees\hunt-base\backend\eval\regression.py:115:     caller can choose to snapshot the
current numbers via `save_baseline`.
.claude\worktrees\hunt-base\backend\eval\regression.py:121:     baselines =
load_baselines(baseline_path)
.claude\worktrees\hunt-base\backend\eval\regression.py:160:def regress_with_provider(db:
Database, video_id: str, video_ref: str,
.claude\worktrees\hunt-base\backend\eval\regression.py:178:     return regress(db, video_id, gt,
stage=stage, iou_threshold=iou_threshold,
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:62:     "load_baselines",
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:63:     "save_baseline",

```

```

.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:137: def load_baselines(path: str
= DEFAULT_PER_USER_BASELINE_PATH) -> Dict[str, dict]:
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:145: def save_baseline(baseline:
PerUserBaseline,
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:149:     Round-trips through
`load_baselines`; returns the key written. Mirrors
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:150:
`regression.save_baseline` (atomic-ish read-modify-write of the one JSON file).
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:152:     baselines =
load_baselines(path)
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:280:     # per-user analogue of
regression.save_baseline. Does not change the decision above.
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:282:         save_baseline(
.claude\worktrees\hunt-base\docs\GOLDEN_SET_IMPLEMENTATION.md:22:
regress(video, tier) (GS-2) compares the two
.claude\worktrees\hunt-base\docs\GOLDEN_SET_IMPLEMENTATION.md:34: | **GS-2** | `feat/regression-
runner` | `regress(video, tier, tolerance)` over existing harness + `baseline.json`
snapshot/compare + CLI | GS-1 | No |
.claude\worktrees\hunt-base\docs\GOLDEN_SET_IMPLEMENTATION.md:52: `regress(video, tier="fast",
tolerance=0.05)` : get GT from `annotation_registry.get(tier)`, read predictions via the existing
`eval.harness`, score with the existing `metrics`, compare against a per-video `baseline.json`,
and flag a regression when precision/recall drop beyond tolerance. Pure orchestration over
existing parts + a small baseline store. CLI: `python run_regression.py --video ... --tier
file`.
.claude\worktrees\hunt-base\docs\GOLDEN_SET_IMPLEMENTATION.md:61: Wire `regress()` into CI (pre-
merge gate on pipeline changes), an on-demand CLI (the "surprise regression"), and an optional
scheduled run. Includes a deliberate re-snapshot path so legitimate production improvements can
rebaseline instead of raising false regressions.
.claude\worktrees\hunt-base\docs\EXPERIMENT_HARNESS.md:77: - `regress(db, video_id, ground_truth,
stage="final", iou_threshold=0.5, detector=None, tolerance=DEFAULT_TOLERANCE,
baseline_path=DEFAULT_BASELINE_PATH) -> RegressionResult` (:104) ? score stored preds vs GT,
flag if P **or** R drops > tolerance.
.claude\worktrees\hunt-base\docs\EXPERIMENT_HARNESS.md:78: - `regress_with_provider(...)` (:160)
? same, fetching GT via the annotation provider.
.claude\worktrees\hunt-base\docs\EXPERIMENT_HARNESS.md:79: - `save_baseline(video_id, stage,
precision, recall, f1, ..., gt_fingerprint=None)` (:80) · `load_baselines(path) -> dict` (:68).
.claude\worktrees\hunt-
base\docs\EXPERIMENT_HARNESS.md:111: `archives/research/AUDIO_RALLY_DETECTION_PLAN.md`. The
`regress()` + baseline path is the canonical comparison mechanism.
.claude\worktrees\hunt-base\docs\EXPERIMENT_HARNESS.md:153: `regression.regress()` +
`baseline.json` + `run_regression.py` exit codes. A variant becomes the new
.claude\worktrees\hunt-base\docs\GOLDEN_REGRESSION_FIXTURES.md:82: [Omitted long matching line]
.claude\worktrees\hunt-base\docs\GOLDEN_SET_VENDORING.md:8: [Omitted long matching line]
.claude\worktrees\hunt-base\docs\GOLDEN_SET_VENDORING.md:108: def regress(video, tier="fast",
tolerance=0.05):
.claude\worktrees\hunt-base\docs\GOLDEN_SET_VENDORING.md:121: **Triggers** (all feed the same
`regress()`): CI on pipeline changes · on-demand (the "surprise" case) · scheduled cron · pre-
release gate.
.claude\worktrees\hunt-base\docs\GOLDEN_SET_VENDORING.md:177: | 3 | `AutomatedProvider`
(independent oracle) + `regress()` + baseline snapshots | Phase 2 |
.claude\worktrees\hunt-base\docs\GOLDEN_SET_VENDORING.md:182: [Omitted long matching line]
.claude\worktrees\hunt-base\tests\test_eval_regression.py:1: """Tests for the regression runner
(GS-2): baseline store, regress(), the
.claude\worktrees\hunt-base\tests\test_eval_regression.py:27:
regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
.claude\worktrees\hunt-base\tests\test_eval_regression.py:28:     b =
regression.load_baselines(p)
.claude\worktrees\hunt-base\tests\test_eval_regression.py:32: def
test_load_baselines_absent_is_empty(tmp_path):
.claude\worktrees\hunt-base\tests\test_eval_regression.py:33:     assert
regression.load_baselines(str(tmp_path / "nope.json")) == {}
.claude\worktrees\hunt-base\tests\test_eval_regression.py:36: # ----- regress() -----
.claude\worktrees\hunt-base\tests\test_eval_regression.py:39:     r = regression.regress(db,
"vid1", GT, baseline_path=str(tmp_path / "b.json"))
.claude\worktrees\hunt-base\tests\test_eval_regression.py:47:

```

```

regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
.claude\worktrees\hunt-base\tests\test_eval_regression.py:48:    r = regression.regress(db,
"vid1", GT, baseline_path=p)
.claude\worktrees\hunt-base\tests\test_eval_regression.py:58:
regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
.claude\worktrees\hunt-base\tests\test_eval_regression.py:60:    r = regression.regress(d,
"vid1", GT, tolerance=0.05, baseline_path=p)
.claude\worktrees\hunt-base\tests\test_eval_regression.py:66:def
test_within_tolerance_does_not_regress(tmp_path):
.claude\worktrees\hunt-base\tests\test_eval_regression.py:73:
regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
.claude\worktrees\hunt-base\tests\test_eval_regression.py:74:    r = regression.regress(d,
"vid1", GT, tolerance=0.05, baseline_path=p)
.claude\worktrees\hunt-base\tests\test_eval_regression.py:80:def
test_regress_with_provider_file_tier(db, tmp_path):
.claude\worktrees\hunt-base\tests\test_eval_regression.py:83:    r =
regression.regress_with_provider(
.claude\worktrees\hunt-base\tests\test_eval_regression.py:115:
regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=base)
.claude\worktrees\hunt-base\tests\test_eval_regression.py:147:
regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p, iou_threshold=0.5,
.claude\worktrees\hunt-base\tests\test_eval_regression.py:150:    r = regression.regress(db,
"vid1", GT, iou_threshold=0.3, baseline_path=p)
.claude\worktrees\hunt-base\tests\test_eval_regression.py:157:
regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p, iou_threshold=0.5,
.claude\worktrees\hunt-base\tests\test_eval_regression.py:159:    r = regression.regress(db,
"vid1", [(0.0, 10.0), (21.0, 31.0)], iou_threshold=0.5, baseline_path=p)
.claude\worktrees\hunt-base\tests\test_automated_provider.py:89:    r =
regression.regress_with_provider(
.claude\worktrees\hunt-base\tests\test_per_user_regression.py:22:    load_baselines,
.claude\worktrees\hunt-base\tests\test_per_user_regression.py:23:    save_baseline,
.claude\worktrees\hunt-base\tests\test_per_user_regression.py:135:    key = save_baseline(rec,
path=path)
.claude\worktrees\hunt-base\tests\test_per_user_regression.py:138:    loaded =
load_baselines(path)
.claude\worktrees\hunt-base\tests\test_per_user_regression.py:147:    save_baseline(rec2,
path=path)
.claude\worktrees\hunt-base\tests\test_per_user_regression.py:148:    both =
load_baselines(path)
.claude\worktrees\hunt-base\tests\test_per_user_regression.py:160:    saved =
load_baselines(path)

```

## 6. user (2026-06-18T01:40:42.605Z)

```

eval_baselines\heuristic_lovo_n6.json
eval_baselines\gemini_wrapper_2026-06-10.json
.claude\worktrees\bhi-execpolicy\eval_baselines\experiment_leaderboard.json
.claude\worktrees\bhi-execpolicy\eval_baselines\gemini_wrapper_2026-06-10.json
.claude\worktrees\bhi-execpolicy\eval_baselines\heuristic_lovo_n6.json
eval_baselines\experiment_leaderboard.json
.claude\worktrees\agent-a771ff1d85e18f2ed\eval_baselines\experiment_leaderboard.json
.claude\worktrees\agent-a771ff1d85e18f2ed\eval_baselines\gemini_wrapper_2026-06-10.json
.claude\worktrees\agent-a771ff1d85e18f2ed\eval_baselines\heuristic_lovo_n6.json
.claude\worktrees\local-noai\eval_baselines\experiment_leaderboard.json
.claude\worktrees\local-noai\eval_baselines\gemini_wrapper_2026-06-10.json
.claude\worktrees\local-noai\eval_baselines\heuristic_lovo_n6.json
.claude\worktrees\docs-rally-quality\eval_baselines\experiment_leaderboard.json
.claude\worktrees\docs-rally-quality\eval_baselines\gemini_wrapper_2026-06-10.json
.claude\worktrees\docs-rally-quality\eval_baselines\heuristic_lovo_n6.json
.claude\worktrees\tier0-eval\eval_baselines\experiment_leaderboard.json
.claude\worktrees\tier0-eval\eval_baselines\gemini_wrapper_2026-06-10.json
.claude\worktrees\tier0-eval\eval_baselines\heuristic_lovo_n6.json
.claude\worktrees\tier1-windowing\eval_baselines\experiment_leaderboard.json
.claude\worktrees\tier1-windowing\eval_baselines\gemini_wrapper_2026-06-10.json
.claude\worktrees\tier1-windowing\eval_baselines\heuristic_lovo_n6.json

```

```
.claude\worktrees\w2-netcross\eval_baselines\experiment_leaderboard.json
.claude\worktrees\w2-netcross\eval_baselines\gemini_wrapper_2026-06-10.json
.claude\worktrees\w2-netcross\eval_baselines\heuristic_lovo_n6.json
.claude\worktrees\w2-safeguard\eval_baselines\experiment_leaderboard.json
.claude\worktrees\w2-safeguard\eval_baselines\gemini_wrapper_2026-06-10.json
.claude\worktrees\w2-safeguard\eval_baselines\heuristic_lovo_n6.json
.claude\worktrees\hardchop\eval_baselines\experiment_leaderboard.json
.claude\worktrees\hardchop\eval_baselines\gemini_wrapper_2026-06-10.json
.claude\worktrees\hardchop\eval_baselines\heuristic_lovo_n6.json
.claude\worktrees\docs-today\eval_baselines\experiment_leaderboard.json
.claude\worktrees\docs-today\eval_baselines\gemini_wrapper_2026-06-10.json
.claude\worktrees\docs-today\eval_baselines\heuristic_lovo_n6.json
.claude\worktrees\docs-golden\eval_baselines\experiment_leaderboard.json
.claude\worktrees\docs-golden\eval_baselines\gemini_wrapper_2026-06-10.json
.claude\worktrees\docs-golden\eval_baselines\heuristic_lovo_n6.json
.claude\worktrees\r2-impl\eval_baselines\experiment_leaderboard.json
.claude\worktrees\r2-impl\eval_baselines\gemini_wrapper_2026-06-10.json
.claude\worktrees\r2-impl\eval_baselines\heuristic_lovo_n6.json
.claude\worktrees\r4-endrule\eval_baselines\experiment_leaderboard.json
.claude\worktrees\r4-endrule\eval_baselines\gemini_wrapper_2026-06-10.json
.claude\worktrees\r4-endrule\eval_baselines\heuristic_lovo_n6.json
.claude\worktrees\boxhill17\eval_baselines\experiment_leaderboard.json
.claude\worktrees\boxhill17\eval_baselines\gemini_wrapper_2026-06-10.json
.claude\worktrees\boxhill17\eval_baselines\heuristic_lovo_n6.json
eval_baselines\nightly_baseline.json
.claude\worktrees\pr-stitch\eval_baselines\experiment_leaderboard.json
.claude\worktrees\pr-stitch\eval_baselines\gemini_wrapper_2026-06-10.json
.claude\worktrees\pr-stitch\eval_baselines\heuristic_lovo_n6.json
.claude\worktrees\pr-stitch\eval_baselines\nightly_baseline.json
.claude\worktrees\boxhill17\eval_baselines\nightly_baseline.json
.claude\worktrees\hunt-base\eval_baselines\experiment_leaderboard.json
.claude\worktrees\hunt-base\eval_baselines\gemini_wrapper_2026-06-10.json
.claude\worktrees\hunt-base\eval_baselines\heuristic_lovo_n6.json
.claude\worktrees\hunt-base\eval_baselines\nightly_baseline.json
.claude\worktrees\gap-doc\eval_baselines\experiment_leaderboard.json
.claude\worktrees\gap-doc\eval_baselines\gemini_wrapper_2026-06-10.json
.claude\worktrees\gap-doc\eval_baselines\heuristic_lovo_n6.json
.claude\worktrees\gap-doc\eval_baselines\nightly_baseline.json
```

## 7. assistant (2026-06-18T01:40:46.674Z)

Important observation: the actual baseline files in `eval\_baselines/` are named `heuristic\_lovo\_n6.json`, `gemini\_wrapper\_2026-06-10.json`, `nightly\_baseline.json`, `experiment\_leaderboard.json` ? NOT the `DEFAULT\_BASELINE\_PATH` of `regression\_baseline.json`. Let me check whether `regression\_baseline.json` exists and look at the caller `run\_regression.py`.

## 8. user (2026-06-18T01:40:47.363Z)

```
1      """CLI: regression-test the production tool against golden-set ground truth.
2
3      Obtains ground truth from an annotation provider (a tier), scores the video's
4      stored predictions, and compares to a snapshotted baseline. Exit code 1 on a
5      detected regression so it can gate CI. See docs/GOLDEN_SET_IMPLEMENTATION.md.
6
7      Examples
8      -----
9      Snapshot the current numbers as the baseline (first run / after a real improvement):
10         python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
   --update-baseline
11
12     Check for a regression (exit 1 if precision/recall dropped beyond tolerance):
13         python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
14     """
15     import os
```



```

16     import sys
17     import json
18     import argparse
19     import logging
20
21     from backend.infrastructure.database import Database
22     from backend.config import load_config
23     from backend.annotations import annotation_registry
24     from backend.eval import regression
25
26
27     def _default_db_path() -> str:
28         cfg = load_config()
29         return os.path.join(cfg.output.output_dir, cfg.output.db_name)
30
31
32     def build_parser() -> argparse.ArgumentParser:
33         p = argparse.ArgumentParser(description="Regression-test the tool vs golden-set
ground truth.")
34         p.add_argument("--video-id", required=True, help="Stored video_id (MD5) whose
predictions to score.")
35         p.add_argument("--tier", default="file",
36                         help=f"Annotation provider tier. Available:
{annotation_registry.list_available()}")
37         p.add_argument("--annotations", help="video_ref for the provider (e.g. CSV path for
the 'file' tier).")
38         p.add_argument("--stage", choices=("candidates", "final"), default="final")
39         p.add_argument("--detector", default=None, help="Restrict candidate scoring to one
detector.")
40         p.add_argument("--iou", type=float, default=0.5)
41         p.add_argument("--tolerance", type=float, default=regression.DEFAULT_TOLERANCE)
42         p.add_argument("--db", default=None)
43         p.add_argument("--baseline", default=regression.DEFAULT_BASELINE_PATH,
help="Baseline JSON path.")
44         p.add_argument("--update-baseline", action="store_true",
45                         help="Snapshot the current numbers as the new baseline instead of
checking.")
46         p.add_argument("--allow-missing-baseline", action="store_true",
47                         help="Treat a missing baseline as a pass (exit 0) instead of a gate
error (exit 3).")
48         p.add_argument("--json", dest="json_out", default=None)
49         return p
50
51
52     def main(argv=None) -> int:
53         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
54         args = build_parser().parse_args(argv)
55
56         db_path = args.db or _default_db_path()
57         if not os.path.exists(db_path):
58             print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
file=sys.stderr)
59             return 2
60         db = Database(db_path)
61
62         result = regression.regress_with_provider(
63             db, video_id=args.video_id, video_ref=args.annotations, tier=args.tier,
64             stage=args.stage, iou_threshold=args.iou, detector=args.detector,
65             tolerance=args.tolerance, baseline_path=args.baseline,
66         )
67
68         print(regression.format_result(result))
69
70         if args.update_baseline:
71             regression.save_baseline(result.video_id, result.stage, result.precision,

```

```

72             result.recall, result.f1, path=args.baseline,
73             iou_threshold=result.iou_threshold,
detector=args.detector,
74             gt_fingerprint=result.gt_hash)
75         print(f"Baseline updated in {args.baseline}.")
76
77     if args.json_out:
78         with open(args.json_out, "w") as f:
79             json.dump(result.as_dict(), f, indent=2)
80             print(f"JSON written to {args.json_out}")
81
82     # Exit codes: 1 = regression, 3 = no baseline to compare (gate cannot pass), 0 = ok.
83     if result.regressed:
84         return 1
85     if not result.has_baseline and not args.update_baseline and not
args.allow_missing_baseline:
86         print("ERROR: no baseline to compare against ? the gate cannot verify anything.
"
87             "Snapshot one with --update-baseline (then commit it), or pass "
88             "--allow-missing-baseline to accept this run.", file=sys.stderr)
89     return 3
90     return 0
91
92
93 if __name__ == "__main__":
94     raise SystemExit(main())
95

```

## 9. user (2026-06-18T01:40:48.106Z)

No files found

## 10. assistant (2026-06-18T01:40:56.403Z)

The `regression\_baseline.json` (the `DEFAULT\_BASELINE\_PATH`) does not exist in the tracked tree. Baselines are always written by `save\_baseline`, which always includes both `precision` and `recall` keys. Let me check the test file to confirm there's no test exercising a malformed baseline, and verify the candidate's premise about the file being "checked-in".

## 11. user (2026-06-18T01:40:57.236Z)

```

1     """Tests for the regression runner (GS-2): baseline store, regress(), the
2     provider-driven end-to-end path, and the CLI exit-code contract."""
3     import pytest
4
5     from backend.infrastructure.database import Database
6     from backend.eval import regression
7     import run_regression
8
9
10    @pytest.fixture
11    def db(tmp_path):
12        d = Database(str(tmp_path / "t.db"))
13        d.add_video("vid1", "/x/m.mp4", "processed", [])
14        # Two confirmed segments matching the GT below ? perfect baseline.
15        d.add_play_segment("vid1", 0.0, 10.0)
16        d.add_play_segment("vid1", 20.0, 30.0)
17        return d
18
19
20    GT = [(0.0, 10.0), (20.0, 30.0)]
21
22
23    # ----- baseline store -----

```

```

24
25 def test_save_and_load_baseline_roundtrip(tmp_path):
26     p = str(tmp_path / "b.json")
27     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
28     b = regression.load_baselines(p)
29     assert b["vid1::final"]["precision"] == 1.0
30
31
32 def test_load_baselines_absent_is_empty(tmp_path):
33     assert regression.load_baselines(str(tmp_path / "nope.json")) == {}
34
35
36 # ----- regress() -----
37
38 def test_no_baseline_is_not_a_regression(db, tmp_path):
39     r = regression.regress(db, "vid1", GT, baseline_path=str(tmp_path / "b.json"))
40     assert r.regressed is False
41     assert r.precision == 1.0 and r.recall == 1.0
42     assert any("no baseline" in why for why in r.reasons)
43
44
45 def test_matching_baseline_passes(db, tmp_path):
46     p = str(tmp_path / "b.json")
47     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
48     r = regression.regress(db, "vid1", GT, baseline_path=p)
49     assert r.regressed is False
50
51
52 def test_recall_drop_beyond_tolerance_regresses(tmp_path):
53     # DB has only ONE of the two GT rallies ? recall 0.5 vs baseline 1.0.
54     d = Database(str(tmp_path / "t.db"))
55     d.add_video("vid1", "/x/m.mp4", "processed", [])
56     d.add_play_segment("vid1", 0.0, 10.0)
57     p = str(tmp_path / "b.json")
58     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
59
60     r = regression.regress(d, "vid1", GT, tolerance=0.05, baseline_path=p)
61     assert r.regressed is True
62     assert r.recall == pytest.approx(0.5)
63     assert any("recall" in why for why in r.reasons)
64
65
66 def test_within_tolerance_does_not_regress(tmp_path):
67     d = Database(str(tmp_path / "t.db"))
68     d.add_video("vid1", "/x/m.mp4", "processed", [])
69     d.add_play_segment("vid1", 0.0, 10.0)
70     d.add_play_segment("vid1", 20.0, 30.0)
71     p = str(tmp_path / "b.json")
72     # Baseline only 0.02 above current perfect score ? within default 0.05 tolerance.
73     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
74     r = regression.regress(d, "vid1", GT, tolerance=0.05, baseline_path=p)
75     assert r.regressed is False
76
77
78 # ----- provider-driven end-to-end (uses the GS-1 FileProvider) -----
79
80 def test_regress_with_provider_file_tier(db, tmp_path):
81     csv = tmp_path / "gt.csv"
82     csv.write_text("start,end\n0,10\n20,30\n")
83     r = regression.regress_with_provider(
84         db, video_id="vid1", video_ref=str(csv), tier="file",
85         baseline_path=str(tmp_path / "b.json"),
86     )
87     assert r.precision == 1.0 and r.recall == 1.0
88

```

```

89
90 # ----- CLI exit-code contract (CI gating) -----
91
92 def test_cli_update_baseline_then_clean_run(db, tmp_path, capsys, monkeypatch):
93     # Point the runner at our temp DB via --db; CSV ground truth via 'file' tier.
94     csv = tmp_path / "gt.csv"
95     csv.write_text("start,end\n0,10\n20,30\n")
96     base = str(tmp_path / "b.json")
97     db_path = db.db_path if hasattr(db, "db_path") else str(tmp_path / "t.db")
98
99     # First, snapshot the baseline ? exit 0.
100    rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
101                             "--annotations", str(csv), "--db", db_path,
102                             "--baseline", base, "--update-baseline"])
103    assert rc == 0
104    assert "REGRESSED" not in capsys.readouterr().out
105
106
107    def test_cli_regression_exits_nonzero(tmp_path, capsys):
108        # DB missing one rally vs a baseline of perfect ? regression ? exit 1.
109        d = Database(str(tmp_path / "t.db"))
110        d.add_video("vid1", "/x/m.mp4", "processed", [])
111        d.add_play_segment("vid1", 0.0, 10.0)
112        csv = tmp_path / "gt.csv"
113        csv.write_text("start,end\n0,10\n20,30\n")
114        base = str(tmp_path / "b.json")
115        regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=base)
116
117        rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
118                                 "--annotations", str(csv), "--db", str(tmp_path / "t.db"),
119                                 "--baseline", base])
120        assert rc == 1
121        assert "REGRESSED" in capsys.readouterr().out
122
123
124    def test_cli_missing_baseline_exits_3(db, tmp_path):
125        # No baseline + no --update/--allow ? the gate cannot verify ? exit 3 (not a silent
126        pass).
127        csv = tmp_path / "gt.csv"
128        csv.write_text("start,end\n0,10\n20,30\n")
129        db_path = str(tmp_path / "t.db")
130        rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
131                                 "--annotations", str(csv), "--db", db_path,
132                                 "--baseline", str(tmp_path / "absent.json")])
133        assert rc == 3
134
135
136    def test_cli_missing_baseline_allowed_exits_0(db, tmp_path):
137        csv = tmp_path / "gt.csv"
138        csv.write_text("start,end\n0,10\n20,30\n")
139        rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
140                                 "--annotations", str(csv), "--db", str(tmp_path / "t.db"),
141                                 "--baseline", str(tmp_path / "absent.json"),
142                                 "--allow-missing-baseline"])
143        assert rc == 0
144
145
146    def test_iou_mismatch_is_incommensurable(db, tmp_path):
147        p = str(tmp_path / "b.json")
148        regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p, iou_threshold=0.5,
149                                gt_fingerprint=regression.gt_hash(GT))
150        # Same labels but scored at a different IoU ? not a valid comparison.
151        r = regression.regress(db, "vid1", GT, iou_threshold=0.3, baseline_path=p)
152        assert r.regressed is True
153        assert any("incommensurable" in why for why in r.reasons)

```

```

153
154
155     def test_changed_labels_detected_via_gt_hash(db, tmp_path):
156         p = str(tmp_path / "b.json")
157         regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p, iou_threshold=0.5,
158                                 gt_fingerprint=regression.gt_hash(GT))
159         r = regression.regress(db, "vid1", [(0.0, 10.0), (21.0, 31.0)], iou_threshold=0.5,
baseline_path=p)
160         assert r.regressed is True
161         assert any("labels changed" in why for why in r.reasons)
162

```

## 12. assistant (2026-06-18T01:41:08.265Z)

Now I have a complete picture. Let me assess the candidate against the reachability lens.

The candidate claims line 129 `base_p, base_r = b["precision"], b["recall"]` will crash with `KeyError` if the persisted baseline file contains a record missing `'precision'/'recall'`. Let me evaluate:

**\*\*Reachability analysis:\*\***

The trigger requires a baseline record `b` that lacks `'precision'` or `'recall'` keys. How does a record get into the baseline file?

1. **\*\*`save_baseline()` (line 80-99)\*\*** ? the only code path that writes baseline records ? always writes both `"precision": precision` and `"recall": recall` (line 93). So records written by the codebase always have both keys.

2. **\*\*Hand-editing / external production\*\*** ? the candidate's premise. The `DEFAULT_BASELINE_PATH` is `eval_baselines/regression_baseline.json`. This file does NOT exist in the tracked tree (Glob found no such file). The actual committed baseline files have different names and are consumed by `scripts/nightly_regression.py` (which has its own separate `save_baseline`/`load_baselines`, excluded from this hunt) and the per-user variant ? not by `regression.py::regress()`. So there is no committed `regression_baseline.json` that a maintainer is hand-editing.

The candidate's reachability case rests entirely on a hypothetical: someone hand-edits a baseline file (that doesn't currently exist) and omits a metric key. That's a malformed-input hypothetical, not a concrete trigger via any real code path. `load_baselines` is plain `json.load` with no schema, true ? but the same is equally true for every other field. The asymmetry (sibling fields use `.get()`, these two use `[...]`) is a defensiveness inconsistency, not a bug with a concrete reachable trigger.

Let me confirm there's no other production writer that could produce a partial record (e.g., a migration or a merge tool).

## 13. user (2026-06-18T01:41:10.250Z)

```

.claude\worktrees\hunt-base\backend\eval\calibrate_local.py:90:         print(f" baseline
{BASELINE_LOCAL}: P={b['precision']:.3f} R={b['recall']:.3f} F1={b['f1']:.3f}")
.claude\worktrees\hunt-base\backend\eval\calibrate_local.py:91:         print(f" best-fit
{best_cfg}: P={t['precision']:.3f} R={t['recall']:.3f} F1={t['f1']:.3f} "
.claude\worktrees\hunt-base\backend\eval\calibrate_local.py:118:     ap.add_argument("--metric",
default="f1", choices=["f1", "precision", "recall"])
.claude\worktrees\hunt-base\backend\eval\calibrate_wasb.py:84:     print(f" precision
{fit['baseline']['precision']:.3f} -> {fit['tuned']['precision']:.3f} "
.claude\worktrees\hunt-base\backend\eval\calibration.py:158:         "baseline": {"precision":
b.precision, "recall": b.recall, "f1": b.f1, "mean_iou": b.mean_iou},
.claude\worktrees\hunt-base\backend\eval\calibration.py:159:         "tuned": {"precision":
t.precision, "recall": t.recall, "f1": t.f1, "mean_iou": t.mean_iou},
.claude\worktrees\hunt-base\backend\eval\distill_local.py:131:         print(f" MEAN held-out:
LEARNED P={mean(clf, 'precision'):.3f} R={mean(clf, 'recall'):.3f} "
.claude\worktrees\hunt-base\backend\eval\distill_local.py:132:
f"F1={mean(clf, 'f1'):.3f} | baseline P={mean(base, 'precision'):.3f} "
.claude\worktrees\hunt-base\backend\eval\eval_partial_labels.py:169:

```

```

"precision": sc.precision, "recall": sc.recall, "mean_iou": sc.mean_iou,
.claude\worktrees\hunt-base\backend\eval\eval_partial_labels.py:183:
f"{r['f1']:>6.3f} {r['precision']:>6.3f} {r['recall']:>6.3f} {r['mean_iou']:>6.3f} "
.claude\worktrees\hunt-base\backend\eval\eval_partial_labels.py:187:         f"gate={b['gate']}]
-> F1 {b['f1']:.3f} (P {b['precision']:.3f} / R {b['recall']:.3f})")
.claude\worktrees\hunt-base\backend\eval\experiment.py:31: - the promotion gate stays
`run_regression.py` + `regression_baseline.json`
.claude\worktrees\hunt-base\backend\eval\experiment.py:64: METRICS = ("precision", "recall",
"f1", "mean_iou")
.claude\worktrees\hunt-base\backend\eval\experiment.py:508:     `regression_baseline.json`;
deliberately the size of a baseline file, not a service
.claude\worktrees\hunt-base\backend\eval\fusion_audio.py:95:         f"  shuttle+person      :
F1={a['f1']:.3f} P={a['precision']:.3f} R={a['recall']:.3f}",
.claude\worktrees\hunt-base\backend\eval\fusion_audio.py:96:         f"  shuttle+person+audio :
F1={b['f1']:.3f} P={b['precision']:.3f} R={b['recall']:.3f}",
.claude\worktrees\hunt-base\backend\eval\fusion_compare.py:71:         f"  shuttle-only :
F1={a['f1']:.3f} P={a['precision']:.3f} R={a['recall']:.3f} IoU={a['mean_iou']:.3f}",
.claude\worktrees\hunt-base\backend\eval\fusion_compare.py:72:         f"  +person      :
F1={b['f1']:.3f} P={b['precision']:.3f} R={b['recall']:.3f} IoU={b['mean_iou']:.3f}",
.claude\worktrees\hunt-base\backend\eval\fusion_compare.py:73:         f"  ?F1 = {d['f1']:+.3f}
({res['delta_pct']['f1']:+.1f}%) ?P={d['precision']:+.3f} ?R={d['recall']:+.3f}",
.claude\worktrees\hunt-base\backend\eval\golden_fixtures.py:383:         "precision":
_round(sc.precision),
.claude\worktrees\hunt-base\backend\eval\golden_fixtures.py:939:     float_fields =
("iou_threshold", "precision", "recall", "f1", "mean_iou",
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:4: ``regression_baseline.json``):
a candidate is gated so a global change can never silently
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:37:saved/loaded as JSON, a
sibling to `regression.py`'s ``regression_baseline.json`` approach.
.claude\worktrees\hunt-base\backend\eval\rally_seq_proto.py:141:         "f1":
sc.f1, "precision": sc.precision, "recall": sc.recall,
.claude\worktrees\hunt-base\backend\eval\rally_seg_eval.py:65:         "f1": sc.f1, "precision":
sc.precision, "recall": sc.recall, "mean_iou": sc.mean_iou,
.claude\worktrees\hunt-base\backend\eval\rally_seg_eval.py:82:         "precision",
"recall",
.claude\worktrees\hunt-base\backend\eval\regression.py:33:DEFAULT_BASELINE_PATH =
os.path.join("eval_baselines", "regression_baseline.json")
.claude\worktrees\hunt-base\backend\eval\regression.py:93:         "precision": precision,
"recall": recall, "f1": f1,
.claude\worktrees\hunt-base\backend\eval\regression.py:129:         base_p, base_r =
b["precision"], b["recall"]
.claude\worktrees\hunt-base\backend\eval\tolerance_metrics.py:268:         "precision":
precision, "recall": recall, "f1": f1,

```

#### 14. assistant (2026-06-18T01:41:16.845Z)

I have confirmed the full picture. The only writer of `regression\_baseline.json` records consumed by `regress()` is `save\_baseline()`, which always emits both `precision` and `recall`. The file itself doesn't currently exist in the tree, and no migration/merge code produces partial records.

This is a defensiveness inconsistency, not a reachable defect: the trigger requires hand-editing a (currently nonexistent) baseline file to omit a metric key ? a malformed-input hypothetical, not a concrete path through real code. The candidate itself rates confidence 0.5 and severity low, and explicitly frames it as "could be hand-edited."

Verdict: not a real bug under the reachability lens. KeyError on hand-corrupted input is arguably the \*correct\* fail-loud behavior for a CI gate (silently treating a corrupt baseline as "no baseline" would be worse ? it would let the gate pass on garbage). The asymmetry with sibling `.get()` calls is a code-consistency nit, not a defect a maintainer would file.

#### 15. user (2026-06-18T01:41:28.985Z)

Structured output provided successfully

## 16. assistant (2026-06-18T01:41:37.021Z)

Verdict: not a real bug. Under the reachability lens, no concrete path reaches the KeyError.

Key findings (all in `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base/`):

- `backend/eval/regression.py:129` ? the indexed access `b["precision"], b["recall"]`. The only writer of records consumed here is `save\_baseline()` at `backend/eval/regression.py:80-99`, which unconditionally emits both keys (line 93). No migration/merge/alternate producer of partial records exists.
- `backend/eval/regression.py:33` ? `DEFAULT\_BASELINE\_PATH = eval\_baselines/regression\_baseline.json`. That file does not exist in the tracked tree; the committed `eval\_baselines/\*.json` files are consumed by other modules (`scripts/nightly\_regression.py`, `per\_user\_regression.py`), not by `regress()`.
- `run\_regression.py:62-74` ? the sole caller; always feeds baselines written via `save\_baseline`.
- `tests/test\_eval\_regression.py` ? no test constructs a metric-less baseline record.

The reported `.get()`-vs-`[...]` asymmetry against sibling fields (`backend/eval/regression.py:132`) is a defensiveness inconsistency, not a defect with a concrete trigger. The trigger requires hand-editing a nonexistent baseline file to drop a metric key ? a malformed-input hypothetical. For a CI gate, fail-loud on a corrupt baseline is also arguably correct behavior, since silently treating it as "no baseline" (the proposed fix) would let the gate pass against a garbage reference.